

Hallucide — Design (PoC)

Hackathon "IA et Hallucination", défi Assemblée nationale. Porteur : Hamza Konte.

Nom

Hallucide : *hallucination* + *-cide* (tuer, comme pesticide) + écho *lucide* (lucidité). Le tueur d'hallucinations qui rend l'IA lucide. Vérifié libre sur " AI".

Problème (consigne)

Les IA génératives produisent des réponses inexactes ou non sourcées, et hallucinent jusqu'à leurs propres justifications : une illusion de fiabilité sans ancrage réel. En contexte institutionnel, distinguer une info vérifiée d'un contenu généré est difficile.

Positionnement (assumé)

Hallucide est une **couche de confiance par détection + annotation**, fidèle à la consigne ("la réponse n'est affichée qu'annotée"). Il **ne prévient pas** la génération d'hallucinations et **ne corrige pas** le texte. Ce qu'il tue = **l'illusion de fiabilité** : toute affirmation non ancrée est visiblement marquée 🟡/🔴, donc ne peut plus se faire passer pour un fait. L'utilisateur ne reçoit jamais une hallucination estampillée "vraie". C'est le sens du "-cide" : on tue la fausse fiabilité, pas la sortie brute. **Hors scope (différé)** : correction/remplacement par le fait officiel, suppression d'affirmations, prévention par RAG. (RAG existe en germe via le tool query proactif, mais la démo veut montrer la détection d'une hallucination, pas l'éviter.)

Solution

Un **middleware de confiance** entre l'IA et l'utilisateur. La réponse est interceptée avant affichage, découpée en affirmations atomiques, puis passée par deux couches complémentaires, et n'est affichée qu'annotée : **vérifié / inféré / incertain**.

Thèse centrale : les deux couches sont complémentaires, pas redondantes. -
Les **logprobs** captent l'invention *instable* (le modèle hésite en écrivant). -

Seule la **source datée** démasque les erreurs *apprises confiantes* et les faits obsolètes (le modèle est sûr mais faux ou périmé).

Décisions verrouillées

- **Scope** : PoC (preuve de concept), slice fonctionnelle la plus rapide. Pas production.
- **Modèle** : Mistral **local** (pas de cloud). Vrais logprobs. Narratif IA souveraine on-prem.
- **Hardware** : VPS **CPU-only** — AMD Ryzen 9 3900x, 4 vCores @ 4.6GHz, 32 Go RAM DDR4, 300 Go NVMe, 750 Mbps. **Pas de GPU** → mistral-inference et vLLM exclus (tous deux GPU/CUDA obligatoire).
- **Runtime** : **llama.cpp** (moteur CPU). Charge un modèle **Mistral GGUF**, expose les **logprobs** (`n_probs` sur `/completion`, ou `logprobs/top_logprobs` sur l'endpoint OpenAI-compatible `/v1/chat/completions`). Backend FastAPI parle OpenAI-compatible au serveur llama.cpp local.
- Distinction clé : **modèle** (Mistral) et **runtime/moteur** (llama.cpp) sont orthogonaux. llama.cpp ≠ Llama : c'est un moteur CPU qui charge des poids Mistral. Le modèle reste 100% Mistral.
- **Modèle retenu** : **Mistral-7B-Instruct GGUF Q4_K_M** (~4,5 Go RAM, ~5-10 tok/s sur ce CPU) — compromis vitesse/qualité pour la démo. Alternatives qui tiennent en 32 Go : Mistral-8B Q4, ou Mistral-Small-24B Q4 (~14 Go, ~2-3 tok/s, plus lent).
- **Couche 2 (sources)** : **serveur MCP tricoteuses (moulineuse) interrogé directement**. Endpoint `https://mcp.code4code.eu/mcp` (Streamable HTTP, pas d'auth, testé OK le 2026-06-22). On NE télécharge PAS l'open data et on ne maintient AUCUN SQLite local : le backend est **client MCP** et interroge la donnée à la volée.
- Outils MCP utilisés : `search_recipes/get_recipe` (recettes métier, **commencer par là**), `search_legal_texts` (FTS Typesense), `query_sql` (SQL paramétré sur la base canutes : schémas `assemblee/senat/annuaire/droits_*`), `list_parlement_items/get_parlement_item` (API Parlement + résumés IA). Introspection : `list_tables/describe_table/get_json_schemas`.
- **Avantages** : zéro parsing/ingestion, données à jour temps réel, couverture large (Assemblée + Sénat + Légifrance). 14e législature et +.
- **Coût** : dépendance live à un serveur communautaire pendant la démo → prévoir un **cache court** en mémoire et un repli si réseau coupé.
- **Archi générique à connecteurs** : interface `SourceConnector` ; une impl concrète = `TricoteusesMCP`. D'autres connecteurs (Légifrance direct, etc.) branchables plus tard sans toucher au pipeline.
- **Backend** : Python FastAPI. Client modèle local (OpenAI-compatible) + client MCP.

- **Frontend** : Vite + React, **DSFR (Système de Design de l'État français)** : Marianne, bleu République #000091, composants officiels. Pas de design flompt.

Architecture / flux

Décision clé — génération-en-affirmations (single-pass). On NE génère PAS une prose puis un re-découpage : les affirmations re-générées sont des paraphrases, leurs tokens ne se remappent pas sur les logprobs d'origine (couche 1 cassée). À la place, le modèle local répond **directement sous forme de tableau JSON d'affirmations atomiques**. Chaque affirmation est ainsi composée de **ses propres tokens**, dont les logprobs sont natifs → aucun ré-alignement, le risque le plus dur disparaît. La consigne ("signaler les passages peu fiables") est satisfaite : chaque affirmation est un passage.



Repli si la génération-en-JSON dégrade trop la qualité des réponses : prose + découpage déterministe en phrases avec suivi des offsets de tokens (granularité plus grossière, alignement plus simple que le re-découpage abstraitif).

Composants (unités isolées, interface claire)

Unité	Rôle	Entrée → Sortie	Dépend de
llm_client	Génère en JSON + expose logprobs	prompt → claims[] (texte + logprobs natifs par affirmation)	runtime local OpenAI- compat
confidence_layer	Couche 1	claim + ses logprobs → score [0,1] + flag instable	—
source_layer	Couche 2 (interface SourceConnector)	claim → {statut: vérifié/ faux/aucune, source datée}	TricoteusesMCP (client MCP)
mcp_client	Appelle les tools tricoteuses	requête typée → résultat officiel	https:// mcp.code4code.eu/ mcp
merger	Fusion des 2 couches	claim + score + verdict source → statut final	—
api (FastAPI)	Orchestration	question → réponse annotée (JSON)	tout ci-dessus
frontend (React/ DSFR)	Affichage annoté	JSON → UI colorée + sources	api

Règle de fusion (merger)

Principe dur : les logprobs ne décident JAMAIS vrai/faux. Seule la source juge. Raison : logprobs bruités (valeur basse = souvent choix de synonyme, pas doute factuel) et une hallucination *confiante* score haut. Les logprobs servent à : (a) **cibler/trier** quelles affirmations vérifier en priorité, (b) **qualifier le résidu non-sourçable**, (c) **afficher** un badge "modèle instable ici".

Verdict source	Logprobs	Statut	Rôle logprobs
confirme	peu importe	● vérifié	ignoré
contredit	peu importe	● faux	ignoré

Verdict source	Logprobs	Statut	Rôle logprobs
hors bdd / aucune source	haute conf	🟡 inféré (non vérifiable)	qualifie le résidu
hors bdd / aucune source	basse conf	🔴 probablement inventé	qualifie le résidu

→ Vrai/faux = 100% source. Les logprobs ne touchent le label QUE sur le résidu non-sourçable, + ciblage + affichage.

Score global de confiance (pilote par les verdicts source, pas les logprobs).
Indice pondéré **par statut ET par longueur** de chaque affirmation :

$$\text{score} = \frac{\sum (\text{poids}(\text{statut}_i) \cdot \text{longueur}_i)}{\sum \text{longueur}_i}$$

poids **vérifié 1, inféré 0.3, incertain 0.1, faux 0** ; longueur_i = nb de tokens de l'affirmation. La pondération par longueur évite qu'un petit fragment faux (ex: 5 % du texte) effondre tout le score. Exemple mock (tokens ≈ 13/22/9/8) :
(1·13+0·22+0.3·9+0.1·8)/52 ≈ **32 %**. Le crédit partiel reconnaît l'inféré au-dessus de l'incertain, le faux à zéro. La gravité d'un faux reste signalée à part (alerte rouge). Les arcs du donut sont aussi proportionnels à la longueur.

Confiance du modèle (barre, par affirmation) : $\exp(\text{mean logprob})$ (moyenne géométrique des probas tokens), **pondérée sur les tokens factuels** (entités, nombres, dates, position de vote), optionnellement affinée par l'entropie des top_logprobs aux tokens-clés ($1 - H/H_{\text{max}}$). Signal de stabilité interne, jamais un verdict.

Données : accès via MCP, routage

Source de vérité = le MCP tricoteuses (base canutes, PostgreSQL + Typesense, exposée par les tools). On n'a ni format brut à parser, ni schéma local à maintenir : on interroge `query_sql` (SQL paramétré sur les schémas `assemblee/senat/annuaire`), `search_legal_texts` (FTS) et les recettes métier. Les données sous-jacentes restent les scrutins, votes, députés, mandats, amendements (14e légis. et +).

Vitesse. Le goulot = génération LLM CPU (secondes), pas la source. L'appel MCP ajoute une latence réseau (quelques centaines de ms) → **cache court en mémoire** sur les requêtes répétées d'une même session.

Routage (arbre de décision). La génération single-pass émet chaque affirmation déjà typée + entités extraites : `{text, type, entities: {depute?, scrutin?, position?, attribut?}, logprobs}`. Le SourceConnector route sur type vers le bon tool MCP :

```
vote      → query_sql : résoudre député + scrutin → position de vote
→ comparer
```

```
identite → query_sql : député → attribut (groupe/circo) → comparer
texte/loi → search_legal_texts (FTS Typesense)
autre → pas de source → "incertain" (couche logprobs qualifie)
```

GraphRAG : rejeté. Conçu pour extraire (via LLM) un graphe depuis du **texte non structuré** + sensemaking global. Les données parlementaires sont **déjà un graphe structuré officiel** (rien à extraire) et nos questions sont des lookups factuels précis. Pire : faire bâtir le graphe par un LLM **réintroduit l'hallucination** qu'Hallucide tue. Règle : donnée structurée → requête directe (SQL via MCP) ; vector/graph seulement pour du texte libre, en dernier recours.

Interfaces externes — serveur MCP = contrat de vérification (BONUS build, cœur conceptuel)

Principe d'archi : une seule fonction de vérification, deux entrées. Le pipeline est une **fonction pure** `verify(text, logprobs) → annotated`, appelée à l'identique par la boucle interne ET le serveur MCP. Pas de duplication, pas de "mode externe".

```
verify(text, logprobs):
    claims = split(text, logprobs)          # affirmations + leurs
    logprobs
    for c in claims:
        c.confidence = layer1(c.logprobs)    # couche 1
        c.source     = ground_in_our_db(c)    # couche 2 – NOS bdd
    officielles
        c.statut     = merge(c.confidence, c.source)
    return annotated(claims)
```

- **Entrée interne** : Mistral local génère → `(text, logprobs) → verify`.
- **Entrée MCP** : IA tierce passe `(result, logprobs) → même verify`.
Comme si on avait prompté nous-mêmes. Résout le caveat logprobs (le tiers les fournit).

Contrat MCP `verify` — params essentiels = **(result, logprobs)** :

```
verify({ result, logprobs })
→ { claims: [{ text, confidence, source:{grounded, official_source:
{url,date}|null}, statut }],
    display_allowed }
```

`cited_sources` = **optionnel** (discipline anti-affirmation-sans-source + ciblage de la recherche). Le grounding réel = toujours NOS bdd, jamais la source fournie.

Deux rôles MCP : - `verify(result, logprobs)` — **réactif** : vérifier ce que l'IA a déjà produit. - `query(...)` — **proactif** : fournir les faits officiels datés à l'IA pour qu'elle **n'hallucine pas dès le départ** (ground avant de parler).

```
query("comment a voté le député X sur le scrutin Y ?")  
→ { answer, official_source:{url, date}, confidence }
```

Forme = **langage naturel** (simple pour l'IA), réutilise l'extraction d'entités du pipeline → SQL paramétré interne. Variante structurée dispo : `query({type, depute?, scrutin?})`. **Sécu : jamais de SQL brut exposé** au tiers (injection) — `query` est typé/NL, SQL paramétré en interne uniquement.

Autres tools : `check_vote(depute, scrutin)`, `get_depute(nom)`, `search_scrutins(sujet)`.

Deux règles dures : 1. **On ne fait JAMAIS confiance aux sources fournies.**

L'IA hallucine ses justifications (cf. consigne). La source alléguée = indice (où chercher), pas preuve. **Le juge = la donnée officielle interrogée via le MCP tricoteuses.** Grounding : - fait trouvé + concorde → `grounded=true` →  vérifié (on renvoie NOTRE url officielle datée) - fait trouvé + contredit →  faux (refus d'affichage) - hors couverture bdd →  incertain vérifié ⇔

`grounded==true` (reconfirmé sur l'officiel), jamais juste "source citée". Aucune source → incertain. 2. **Logprobs captables seulement au niveau**

orchestrateur (l'app qui appelle le LLM récupère les logprobs de l'API et les transmet). Pas un tool-call autonome d'un LLM nu. Hallucide se branche dans le pipeline de réponse de l'app hôte = la place d'un middleware.

Pourquoi accepter des sources (optionnelles) si on re-vérifie tout : (a)

discipline (affirmation sans source = signal faible de plus), (b) **ciblage** (l'indice accélère la couche 2). Confirmation = toujours la nôtre, sur nos bdd.

Ordre de build : bonus, après la boucle interne + UI. Mais conceptuellement c'est le cœur d'Hallucide.

Vérification temporelle des sources

Les faits sont indexés dans le temps (consigne : "seule la source datée démasque les faits obsolètes"). Règles : - **Identité = ID stables, jamais le titre** : scrutin → `numero/uid`, député → `uid` (PA...), + date. Le titre ne sert qu'à l'entrée floue (FTS5), puis résolution immédiate vers l'ID. - **Fait-événement** ("a voté pour Y") → scrutin unique daté. **Fait-état** ("est au groupe GDR") → vérifié contre l'état valide à la date de référence (mandats datés). - La décompose extrait la temporalité : `claim = {text, type, entities, date_ref: date|null}`. - **Règle de résolution** :

```
lookup(claim) → enregistrements correspondants + dates
  1 match → vérifier
  N matches + date_ref → enreg. valide à date_ref (état) /
dont date==date_ref (événement)
  N matches + pas de date_ref → concordent → ● ; divergent → ●
ambigu + afficher candidats datés
  0 match → ● incertain
```

- **Éthique (anti-hallucination appliqué à nous)** : si on ne peut pas épingler un enregistrement **unique et daté**, on ne tamponne PAS "vérifié" → "ambigu" + candidats. On refuse de certifier sa propre certitude.
- "Deux sources concurrentes" n'existe quasi pas : une seule autorité (open data officiel) avec plusieurs enregistrements datés ; ambiguïté interne réglée par scoping temporel + ID. Le cas multi-sources contradictoires = post-PoC (plusieurs connecteurs).

Scénario démo (jury)

Question type : "*Le député X a-t-il voté pour la loi Y ?*" ou "*Qui a déposé l'amendement Z ?*". Le modèle local hallucine un **fait à entité précise** (nom de député inventé, mauvais numéro/date de scrutin) → le MCP Assemblée récupère le vrai vote daté → l'affirmation inventée passe ●, la correcte ●.

La couche source est la pièce maîtresse visible ; la couche logprobs est le garnish ("attrapé avant même de vérifier"). Raison : les logprobs sont bruités (les valeurs basses tracent souvent un choix de synonyme/formulation, pas un doute factuel ; et — c'est la thèse — une hallucination *confiante* score haut). Donc : - Choisir une question où la fabrication est une **entité spécifique** (nom/nombre/date) où l'entropie pique vraiment. - **Pré-tester** que le modèle local quantifié hallucine bien CE fait précis de façon reproductible avant la démo.

Hors scope (YAGNI pour le PoC)

- Pas d'auth utilisateur, pas de multi-tenant.
- Pas de connecteur générique multi-domaines (un seul connecteur : Assemblée via MCP). L'archive à connecteurs est montrée comme interface, pas implémentée en N sources.
- Pas de cache, pas de persistance. Tout en mémoire le temps de la requête.
- Pas de streaming token-par-token dans l'UI (réponse annotée d'un bloc).

Risques connus

- **Débit CPU** : Mistral-7B Q4 sur 4 vCores → ~5-10 tok/s. Réponses de démo courtes pour rester fluide. Pré-charger le modèle au boot (pas de cold start pendant la démo).
- **~~Alignement des spans de logprobs~~** :  résolu par la génération-en-affirmations single-pass (logprobs natifs par affirmation).
- **Bruit des logprobs** : couche 1 peu fiable seule (cf. scénario démo). La couche source porte la démo.
- **Dépendance réseau au MCP tricoteuses** : la couche source dépend d'un serveur communautaire live pendant la démo. Mitigations : cache court en mémoire ; pré-jouer le scénario démo pour réchauffer le cache ; repli gracieux ("source indisponible" plutôt que crash) si réseau coupé.
- **query_sql sur schéma non maîtrisé** : la base canutes a ses propres schémas (assemblee/senat/...). Prévoir `list_tables/describe_table` au setup pour fixer les requêtes, et passer par `search_recipes` (recettes métier) avant d'écrire du SQL à la main.
- **Licence DSFR** : réservée à l'usage de l'État. OK pour un hackathon organisé par l'Assemblée ; ne pas publier publiquement comme si c'était state-endorsed.